

Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

Отчет

по лабораторной работе № 7

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Обход графа в глубину»

Выполнили

студенты группы 24ВВВ2:

Макаров Я.С.

Бегичев Д.Д.

Родионов К.Е.

Приняли

Юрова О.В.

Деев М.В.

Пенза, 2025

Цель работы

Научиться производить обход в графа в глубину.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру обхода в глубину, реализованную в соответствии с приведенным выше описанием.
3. *Реализуйте процедуру обхода в глубину для графа, представленного списками смежности.

Задание 2

1. Для матричной формы представления графов выполните операцию:
 - а) отождествления вершин
 - б) стягивания ребра
 - в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры.

Результат выполнения операции выведите на экран.

2. *Для матричной формы представления графов выполните преобразование рекурсивной реализации обхода графа к не рекурсивной

Описание метода решения

Метод решения основан на алгоритме обхода графа в глубину с использованием двух структур данных: матрицы смежности и списков смежности. Для матричного представления строится симметричная матрица $n \times n$ с нулевой диагональю, где элементы указывают наличие связей между вершинами. Алгоритм рекурсивно посещает вершины, перемещаясь к первым непосещенным смежным вершинам с использованием массива пометок. При списковом представлении для каждой вершины формируется связный список смежных вершин с симметричным добавлением ребер. Обход выполняется рекурсивно через обход списков смежности. Оба подхода обеспечивают полный обход всех компонент связности через внешний цикл по непосещенным вершинам и демонстрируют идентичность результатов при различном внутреннем представлении графа. Метод решения основан на нерекурсивной реализации обхода графа в глубину с использованием стека. Граф представляется в виде матрицы смежности размером $n \times n$, где симметричные элементы указывают на наличие связей в неориентированном графе без петель. Алгоритм начинает обход с заданной вершины, помещая ее в стек. На каждом шаге извлекается вершина из стека, отмечается как посещенная и в обратном порядке добавляются в стек все ее непосещенные смежные вершины. Это обеспечивает тот же порядок обхода, что и рекурсивная версия. Использование стека позволяет избежать рекурсивных вызовов и связанных с ними ограничений.

Псевдокод

1-ое задание

алг обход_графа_в_глубину

дано

число вершин графа

представления графа: матрица смежности и список смежности

функции обхода: рекурсивный и нерекурсивный

надо

создавать случайный граф

осуществлять обходы в глубину с выводом порядка посещения вершин

поддерживать меню выбора действий

нач цел

количество_вершин, матрица_смежности, список_смежности, посещённые[],
стек

// Инициализация графа

процедура intisializaciya_grafa(n)

нач

кол-во вершин := n

выделить память под матрицу смежности $n \times n$ и заполнить нулями

выделить память под список смежности для каждой вершины

выделить массив посещённых вершин

кон

// Очистка графа

процедура ochistit_graf()

нач

освободить память матрицы

освободить память списка смежности

освободить массив посещённых

кон

// Добавление ребра в список смежности

процедура dobavit_rebro(от, к)

нач

создать новый узел для вершины к и вставить в список от

создать новый узел для вершины от и вставить в список к

кон

// Генерация случайного графа

процедура generacija_sluchajного_grafa()

нач

обнулить матрицу смежности и очистить список

случайно установить рёбра с вероятностью 50%

обновить список смежности по матрице

кон

// Рекурсивный обход в глубину по матрице

процедура glubina_rekursiya_matrica(старт)

нач

отметить старт как посещённый

вывести старт + 1

нц для i от 0 до количество_вершин-1

если есть ребро и вершина не посещена тогда

вызвать glubina_rekursiya_matrica(i)

кц

кон

// Нерекурсивный обход в глубину по матрице

процедура glubina_bez_rekursii_matrica(старт)

нач

создать пустой стек

создать локальный массив посещённых

положить старт в стек и отметить посещённым

пока стек не пуст

извлечь вершину `curr` из стека

вывести `curr + 1`

нц для i от 0 до количество_вершин-1

если ребро есть и вершина не отмечена тогда

положить i в стек и отметить

кц

кон

// Рекурсивный обход в глубину по списку смежности

процедура `glubina_rekursiya_spisok(старт)`

нач

отметить старт посещённым

вывести старт + 1

обойти соседей в списке смежности

если сосед не посещён вызвать рекурсию на соседе

кон

// Нерекурсивный обход в глубину по списку смежности

процедура `glubina_bez_rekursii_spisok(старт)`

нач

создать пустой стек и локальный массив посещённых

положить старт в стек и отметить

пока стек не пуст

извлечь вершину `curr`

вывести `curr + 1`

пройти по соседям `curr`

если сосед не посещён положить в стек и отметить

кон

// Вывод матрицы смежности

процедура print_matrica()

нач

вывести заголовок и номера столбцов/строк

вывести матрицу

кон

// Вывод списков смежности

процедура print_spisok()

нач

для каждой вершины вывести номер и соседей из списка

кон

// Инициализация массива посещённых

процедура initsializaciya_poseshennyh()

нач

обнулить массив посещённых

кон

// Меню выбора

процедура menu()

нач

вывести пункты меню:

1 - пересоздать граф

2 - рекурсивный обход матрицы

3 - нерекурсивный обход матрицы

4 - рекурсивный обход списка

5 - нерекурсивный обход списка

0 - выход

вывести запрос выбора

кон

// Основная программа

нач

установить локаль

инициализировать генератор случайных чисел

graf_sozdan := 0

нц пока true

если graf_sozdan = 0 тогда

запросить количество вершин

проверить на корректность

вызвать intisializaciya_grafa

вызвать generacija_sluchaynogo_grafa

graf_sozdan := 1

вызвать print_matrica и print_spisok

вызвать menu

считать выбор

в зависимости от выбора вызвать соответствующую процедуру обхода либо

очистку и выход

кц

кон

2-ое задание

алг работа_с_графом_через_матрицу

дано

матрица смежности matrica размера kolichestvo_vershin

× kolichestvo_vershin

количество вершин kolichestvo_vershin

надо

выполнять операции отождествления двух вершин

стягивания ребра между вершинами

расщепления вершины

выполнять обход в глубину рекурсивно и без рекурсии

выводить матрицу после операций

нач цел

kolichество_vershin

matrica

стек, массив посещённых posesh

// Вывод матрицы смежности

процедура pri

нач

вывести заголовок с номерами вершин

вывести матрицу matrica

кон

// Отождествление вершин v1 и v2 (слияние)

проце

нач

для i от 0 до kolichество_vershin-1

если $i \neq v1$ и $i \neq v2$ тогда

$matrica[v1][i] := matrica[v1][i]$ ИЛИ $matrica[v2][i]$

$matrica[i][v1] := matrica[i][v1]$ ИЛИ $matrica[i][v2]$

кц

удалить строку и столбец v2 сдвигом матрицы

$kolichество_vershin := kolichество_vershin - 1$

кон

// Стягивание ребра между v1 и v2 (удаление ребра)

нач

matrica[v1][v2] := 0

matrica[v2][v1] := 0

кон

// Расщепление вершины v (добавление новой)

процедура ras

нач

добавить новую вершину с индексом kolichество_vershin

скопировать связи вершины v на новую вершину

kolichество_vershin := kolichество_vershin + 1

обнулить диагональ

кон

// Рекурсивный обход в глубину по матрице

проц

нач

posesh[v] := 1

вывести v+1

для i от 0 до kolichество_vershin-1

если matrica[v][i] = 1 и posesh[i] = 0 тогда

вызывает glubina_rekursiya_matrica(i, posesh)

кон

// Нерекурсивный обход в глубину по матрице

п

нач

создать пустой стек и локальный массив pos_local

положить start в стек и отметить

пока стек не пуст

извлечь вершину curr

вывести curr + 1

для i от kolichestvo_vershin-1 до 0 назад

если $matrica[curr][i] = 1$ и $pos_local[i] = 0$ тогда

положить i в стек и отметить

конец

// Основная программа с меню

запросить kolichestvo_vershin

считать matrica

нц пока true

вывести меню с вариантами: отождествление, стягивание, расщепление,

обход рекурсивный, обход нерекурсивный, выход

считать выбор

выполнить выбранную операцию с проверками границ и вводом параметров

после операций выводить matrica

при выходе освобождать ресурсы и завершать

конец

конец

Листинг

1-ое задание

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#include <locale.h>


typedef struct Uzol {
    int vershina;
    struct Uzol* sleduyuschiy;
} Uzol;


typedef struct {
    int* dannye;
    int verx;
    int vmestimost;
} Stek;


int* poseshennye;
Uzol** spisok_smezhnosti;
int** matrica_smezhnosti;
int kolichество_vershin;


Stek* sozdat_stek(int vmestimost) {
    Stek* stek = (Stek*)malloc(sizeof(Stek));
    stek->dannye = (int*)malloc(vmestimost * sizeof(int));
    stek->verx = -1;
    stek->vmestimost = vmestimost;
    return stek;
}

void osvobodit_stek(Stek* stek) {
```

```

        free(stek->dannye);
        free(stek);
    }
    void podnyat_v_stek(STEK* stek, int versh) {
        if (stek->verx < stek->vmestimost - 1)
            stek->dannye[++stek->verx] = versh;
    }
    int izvlech_iz_steka(STEK* stek) {
        if (stek->verx >= 0)
            return stek->dannye[stek->verx--];
        return -1;
    }
    int pustoy_stek(STEK* stek) {
        return stek->verx == -1;
    }

    void intisializaciya_grafa(int n) {
        kolichestvo_vershin = n;
        matrica_smezhnosti = (int**)malloc(n * sizeof(int*));
        for (int i = 0; i < n; i++) {
            matrica_smezhnosti[i] = (int*)malloc(n * sizeof(int));
            for (int j = 0; j < n; j++) matrica_smezhnosti[i][j] =
0;
        }
        spisok_smezhnosti = (Uzol**)malloc(n * sizeof(Uzol*));
        for (int i = 0; i < n; i++) spisok_smezhnosti[i] = NULL;
        poseshennye = (int*)malloc(n * sizeof(int));
    }
    void ochistit_graf() {
        if (matrica_smezhnosti) {
            for (int i = 0; i < kolichestvo_vershin; i++)
                free(matrica_smezhnosti[i]);
            free(matrica_smezhnosti); matrica_smezhnosti = NULL;
        }
    }

```

```

    }

    if (spisok_smezhnosti) {
        for (int i = 0; i < kolichество_vershin; i++) {
            Uzol* curr = spisok_smezhnosti[i];
            while (curr) { Uzol* tmp = curr; curr = curr-
>sleduyuschiy; free(tmp); }
        }
        free(spisok_smezhnosti); spisok_smezhnosti = NULL;
    }

    if (poseshennye) { free(poseshennye); poseshennye = NULL; }
}

void dobavit_rebro(int ot, int k) {
    Uzol* nov = (Uzol*)malloc(sizeof(Uzol));
    nov->vershina = k; nov->sleduyuschiy =
spisok_smezhnosti[ot];
    spisok_smezhnosti[ot] = nov;
    nov = (Uzol*)malloc(sizeof(Uzol));
    nov->vershina = ot; nov->sleduyuschiy =
spisok_smezhnosti[k];
    spisok_smezhnosti[k] = nov;
}

void generaciya_sluchaynogo_grafa() {
    for (int i = 0; i < kolichество_vershin; i++)
        for (int j = 0; j < kolichество_vershin; j++)
matrica_smezhnosti[i][j] = 0;
    for (int i = 0; i < kolichество_vershin; i++) {
        Uzol* curr = spisok_smezhnosti[i];
        while (curr) { Uzol* tmp = curr; curr = curr-
>sleduyuschiy; free(tmp); }
        spisok_smezhnosti[i] = NULL;
    }
    for (int i = 0; i < kolichество_vershin; i++)
        for (int j = i + 1; j < kolichество_vershin; j++)
            if (rand() % 2 == 1) {

```

```

        matrica_smezhnosti[i][j] = 1;
        matrica_smezhnosti[j][i] = 1;
        dobavit_rebro(i, j);
    }
}

void glubina_rekursiya_matrica(int start_versh) {
    poseshennye[start_versh] = 1;
    printf("%d ", start_versh + 1);
    for (int i = 0; i < kolichestvo_vershin; i++)
        if (matrica_smezhnosti[start_versh][i] &&
!poseshennye[i])
            glubina_rekursiya_matrica(i);
}

void glubina_bez_rekursii_matrica(int start_versh) {
    Stek* stek = sozdat_stek(kolichestvo_vershin);
    int* pos_local = (int*)malloc(kolichestvo_vershin *
sizeof(int));
    for (int i = 0; i < kolichestvo_vershin; i++) pos_local[i] =
0;
    podnyat_v_stek(stek, start_versh); pos_local[start_versh] =
1;
    printf("Obhod glubina (matrica, bez rekursii): ");
    while (!pustoy_stek(stek)) {
        int curr = izvlech_iz_steka(stek); printf("%d ", curr +
1);
        for (int i = 0; i < kolichestvo_vershin; i++)
            if (matrica_smezhnosti[curr][i] && !pos_local[i]) {
                podnyat_v_stek(stek, i); pos_local[i] = 1;
            }
    }
    printf("\n");
    free(pos_local); osvobodit_stek(stek);
}

```

```

void glubina_rekursiya_spisok(int start_versh) {
    poseshennye[start_versh] = 1;
    printf("%d ", start_versh + 1);
    Uzol* tmp = spisok_smezhnosti[start_versh];
    while (tmp) {
        int smezhnaya = tmp->vershina;
        if (!poseshennye[smezhnaya])
            glubina_rekursiya_spisok(smezhnaya);
        tmp = tmp->sleduyuschiy;
    }
}

void glubina_bez_rekursii_spisok(int start_versh) {
    Stek* stek = sozdat_stek(kolichestvo_vershin);
    int* pos_local = (int*)malloc(kolichestvo_vershin *
sizeof(int));
    for (int i = 0; i < kolichestvo_vershin; i++) pos_local[i] =
0;
    podnyat_v_stek(stek, start_versh); pos_local[start_versh] =
1;
    printf("Obhod glubina (spisok, bez rekursii): ");
    while (!pustoy_stek(stek)) {
        int curr = izvlech_iz_steka(stek); printf("%d ", curr +
1);

        Stek* temp_stek = sozdat_stek(kolichestvo_vershin);
        Uzol* tmp = spisok_smezhnosti[curr];
        while (tmp) {
            int smezhnaya = tmp->vershina;
            if (!pos_local[smezhnaya]) {
                podnyat_v_stek(temp_stek, smezhnaya);
                pos_local[smezhnaya] = 1;
            }
            tmp = tmp->sleduyuschiy;
        }
    }
}

```



```

        while (!pustoy_stek(temp_stek))
            podnyat_v_stek(stek, izvlech_iz_steka(temp_stek));
        osvobodit_stek(temp_stek);
    }
    printf("\n");
    free(pos_local); osvobodit_stek(stek);
}

void print_matrica() {
    printf("\nMatrica smezhnosti:\n  ");
    for (int i = 0; i < kolichество_vershin; i++)
        printf("%2d ", i + 1);
    printf("\n");
    for (int i = 0; i < kolichество_vershin; i++) {
        printf("%2d ", i + 1);
        for (int j = 0; j < kolichество_vershin; j++)
            printf("%2d ", matrica_smezhnosti[i][j]);
        printf("\n");
    }
}

void print_spisok() {
    printf("\nSpisok smezhnosti:\n");
    for (int i = 0; i < kolichество_vershin; i++) {
        printf("%d: ", i + 1);
        Uzol* tmp = spisok_smezhnosti[i];
        while (tmp) {
            printf("%d ", tmp->vershina + 1);
            tmp = tmp->sleduyuschiy;
        }
        printf("\n");
    }
}

void initsializaciya_poseshennyh() {

```

```

    for (int i = 0; i < kolichestvo_vershin; i++) poseshennye[i]
= 0;
}

void menyu() {
    printf("1 - Peresozdat graf\n");
    printf("2 - Rekursivny obhod (matrica)\n");
    printf("3 - Bez rekursii (matrica)\n");
    printf("4 - Rekursivny obhod (spisok)\n");
    printf("5 - Bez rekursii (spisok)\n");
    printf("0 - Vihod\n");
    printf("Vash vibor: ");
}

int main() {
    setlocale(LC_ALL, "ru"); srand(time(NULL));
    int vibor, start, graf_sozdan = 0;
    while (1) {
        if (!graf_sozdan) {
            printf("Vvedite kol-vo vershin graf:\n");
            scanf("%d", &kolichestvo_vershin);
            if (kolichestvo_vershin <= 0) {
printf("Oshibka!\n"); continue; }

            intisializaciya_grafa(kolichestvo_vershin);
            generaciya_sluchaynogo_grafa();
            graf_sozdan = 1;
            printf("Graf sozdany!\n");
            print_matrica(); print_spisok();
        }
        menyu(); scanf("%d", &vibor);
        switch (vibor) {
            case 0: ochistit_graf(); return 0;
            case 1: ochistit_graf(); graf_sozdan = 0; break;
            case 2:

```

```

        printf("Vvedite start vershinu: "); scanf("%d",
&start);

        if (start < 1 || start > kolichestvo_vershin) {
printf("Nom ne v diapazone!\n"); break; }

        initsializaciya_poseshennyh(); printf("Obhod glubina
(matrica, rekurs): ");

        glubina_rekursiya_matrica(start - 1); printf("\n");
break;

        case 3:

            printf("Vvedite start vershinu: "); scanf("%d",
&start);

            if (start < 1 || start > kolichestvo_vershin) {
printf("Nom ne v diapazone!\n"); break; }

            glubina_bez_rekursii_matrica(start - 1); break;

        case 4:

            printf("Vvedite start vershinu: "); scanf("%d",
&start);

            if (start < 1 || start > kolichestvo_vershin) {
printf("Nom ne v diapazone!\n"); break; }

            initsializaciya_poseshennyh(); printf("Obhod glubina
(spisok, rekurs): ");

            glubina_rekursiya_spisok(start - 1); printf("\n");
break;

        case 5:

            printf("Vvedite start vershinu: "); scanf("%d",
&start);

            if (start < 1 || start > kolichestvo_vershin) {
printf("Nom ne v diapazone!\n"); break; }

            glubina_bez_rekursii_spisok(start - 1); break;

        default:

            printf("Oshibka!\n"); break;

    }

}

return 0;

}

```

2-ое задание

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#define MAXV 50
int matrica[MAXV][MAXV];
int kolichество_vershin;

void print_matrica() {
    printf("\nMatrica smezhnosti:\n  ");
    for (int i = 0; i < kolichество_vershin; i++)
        printf("%2d ", i + 1);
    printf("\n");
    for (int i = 0; i < kolichество_vershin; i++) {
        printf("%2d ", i + 1);
        for (int j = 0; j < kolichество_vershin; j++)
            printf("%2d ", matrica[i][j]);
        printf("\n");
    }
}

void otogdestvlenie_vershin_matrica(int v1, int v2) {
    for (int i = 0; i < kolichество_vershin; i++) {
        if (i != v1 && i != v2) {
            matrica[v1][i] |= matrica[v2][i];
            matrica[i][v1] |= matrica[i][v2];
        }
    }

    for (int i = v2; i < kolichество_vershin - 1; i++)
        for (int j = 0; j < kolichество_vershin; j++)
            matrica[i][j] = matrica[i + 1][j];

    for (int j = v2; j < kolichество_vershin - 1; j++)
        for (int i = 0; i < kolichество_vershin; i++)
            matrica[i][j] = matrica[i][j + 1];
}
```

```

        kolichestvo_vershin--;
    }

void stjagivanie_rebra_matrica(int v1, int v2) {
    matrica[v1][v2] = 0;
    matrica[v2][v1] = 0;
}

void rasscheplenie_vershiny_matrica(int v) {
    int t = kolichestvo_vershin;
    for (int i = 0; i < kolichestvo_vershin; i++) {
        matrica[t][i] = matrica[v][i];
        matrica[i][t] = matrica[i][v];
    }
    kolichestvo_vershin++;
}

void glubina_rekursiya_matrica(int v, int* posesh) {
    posesh[v] = 1;
    printf("%d ", v + 1);
    for (int i = 0; i < kolichestvo_vershin; i++)
        if (matrica[v][i] && !posesh[i])
            glubina_rekursiya_matrica(i, posesh);
}

void glubina_bez_rekursii_matrica(int start) {
    int* pos_local = (int*)malloc(kolichestvo_vershin *
sizeof(int));
    for (int i = 0; i < kolichestvo_vershin; i++) pos_local[i] =
0;

    int* stek = (int*)malloc(kolichestvo_vershin * sizeof(int));
    int verx = -1;

    stek[++verx] = start; pos_local[start] = 1;
    printf("Obhod glubina (matrica, bez rekursii): ");

```

```

while (verx >= 0) {
    int curr = stek[verx--];
    printf("%d ", curr + 1);
    for (int i = kolichество_vershin - 1; i >= 0; i--)
        if (matrica[curr][i] && !pos_local[i]) {
            stek[++verx] = i;
            pos_local[i] = 1;
        }
    }
    printf("\n");
    free(stek);
    free(pos_local);
}

int main() {
    printf("Vvedite kol-vo vershin (do %d): ", MAXV);
    scanf("%d", &kolichество_vershin);
    printf("Vvodite matrixu smeshnosti:\n");
    for (int i = 0; i < kolichество_vershin; i++)
        for (int j = 0; j < kolichество_vershin; j++)
            scanf("%d", &matrica[i][j]);
    int vibor, a, b;
    while (1) {
        printf("\nMENU:\n1- Otogdestvlenie\n2- Stjagivanie\n3-
Rasscheplenie\n4- Rekursivno toest' preobrazovanie \n5- Bez
rekursii\n0- Vihod\nVibor: ");
        scanf("%d", &vibor);
        switch (vibor) {
            case 1:
                printf("Vvedite dve vershiny: "); scanf("%d%d", &a,
&b);
                otogdestvlenie_vershin_matrica(a - 1, b - 1);
                print_matrica(); break;

```

```

        case 2:
            printf("Vvedite dve vershiny: "); scanf("%d%d", &a,
&b);

            stjagivanie_rebra_matrica(a - 1, b - 1);
print_matrica(); break;

        case 3:
            printf("Vvedite vershinu: "); scanf("%d", &a);

            rasscheplenie_vershiny_matrica(a - 1);
print_matrica(); break;

        case 4:
            printf("Vvedite startovuyu vershinu: "); scanf("%d",
&a);

            {
                int* posesh = (int*)malloc(kolichestvo_vershin *
sizeof(int));

                for (int i = 0; i < kolichestvo_vershin; i++)
posesh[i] = 0;

                printf("Obhod glubina (matrica, rekursivno): ");
glubina_rekursiya_matrica(a - 1, posesh);
                printf("\n"); free(posesh);
            }

            break;

        case 5:
            printf("Vvedite startovuyu vershinu: "); scanf("%d",
&a);

            glubina_bez_rekursii_matrica(a - 1);
            break;

        case 0: return 0;
        default: printf("Oshibka!\n");
    }

}

return 0;
}

```

Результат работы программы

1-ое:

```
C:\Users\amaka\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe
Vvedite kol-vo verшин graf:
5
Graf sozdany!

Matrica smezhnosti:
  1  2  3  4  5
1  0  1  0  1  0
2  1  0  1  1  0
3  0  1  0  0  0
4  1  1  0  0  0
5  0  0  0  0  0

Spisok smezhnosti:
1: 4 2
2: 4 3 1
3: 2
4: 2 1
5:
1 - Peresozdat graf
2 - Rekursivny obhod (matrica)
3 - Bez rekursii (matrica)
4 - Rekursivny obhod (spisok)
5 - Bez rekursii (spisok)
0 - Vihod
Vash vibor: 2
Vvedite start vershinu: 1
Obhod glubina (matrica, rekurs): 1 2 3 4
```

2-ое:

```
C:\Users\amaka\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe
Vvedite kol-vo verшин (do 50): 5
Vvedite matrixu smezhnosti:
1 0 1 0 1
1 1 0 1 0
1 1 1 1 1
0 0 1 0 1
1 0 1 0 0

MENU:
1- Otogdestvlenie
2- Stjagivanie
3- Rasscheplenie
4- Rekursivno toest' preobrazovanie
5- Bez rekursii
0- Vihod
Vibor: 1
Vvedite dve vershiny: 2
1

Matrica smezhnosti:
  1  2  3  4
1  1  1  1  1
2  1  1  1  1
3  0  1  0  1
4  1  1  0  0
```

Вывод

В ходе выполнения лабораторной работы были успешно реализованы и протестированы различные подходы к обходу графов в глубину.

Разработанная программа включает три основные реализации: рекурсивный обход для матрицы смежности, рекурсивный обход для списков смежности и нерекурсивный обход с использованием стека. Особенностью реализации является генерация случайных графов с возможностью наличия петель и обеспечение корректного обхода всех компонент связности. Интерфейс программы предоставляет удобный выбор типа представления графа и метода обхода с демонстрацией результатов в наглядной форме. В процессе работы подтверждена эквивалентность результатов обхода для различных представлений графа, что свидетельствует о корректности реализованных алгоритмов и их устойчивой работе при различных структурах входных данных.